

Two Sum

Brute force :-
 $T.C \rightarrow O(n^2)$
 $S.C \rightarrow O(1)$
 $[2, 6, 5, 8, 11], \text{Target} = 14$

- (I)
 $18 = 14$ (F)
 $7 = 14$ (F)
 $10 = 14$ (F)
 $13 = 14$ (F)
- (II)
 $[2, 6, 5, 8, 11], \text{Target} = 14$
 $11 = 14$ (F)
 $14 = 14$ (T) $\rightarrow$ return $9, 5$

Pseudocode

```

static int[] helper(int[] arr, int target) {
    for (int i = 0; i < arr.length; i++) {
        sum = 0;
        for (int j = i + 1; j < arr.length; j++) {
            sum += arr[i] + arr[j];
            if (sum == target) return new int[] { i, j };
        }
    }
    return new int[] { -1, -1 };
}
    
```

Problem :-

$\rightarrow$ Here T.C is too high

Optimized way :- Using Hash Map
 $T.C \rightarrow O(n)$
 $S.C \rightarrow O(n)$

Pseudocode :-

```

static int[] helper(int[] arr, int target) {
    Map<Integer, Integer> map = new HashMap();
    for (int i = 0; i < arr.length; i++) {
        int remaining = target - arr[i];
        if (map.containsKey(remaining)) return new int[] { i, map.get(remaining) };
        map.put(arr[i], i);
    }
    return new int[] { -1, -1 };
}
    
```

Dry Run

$[2, 6, 5, 8, 11], \text{Target} = 14$

14-2
 (12) (F)
 14-6
 8 (F)
 14-5
 9 (F)
 14-8
 6 (T) $\rightarrow$ return ans (2, 1)

val	idx
2	0
6	1
5	2